Сравнение микросервисов и модульного монолита SmartCity

Лабораторная работа. Авторы: **Ашроев Евгений, Шилов Данил, Белоножко Иван**. Дата проверки и измерений: 27 октября 2026 года. Репозиторий: SmartCityMonolith.

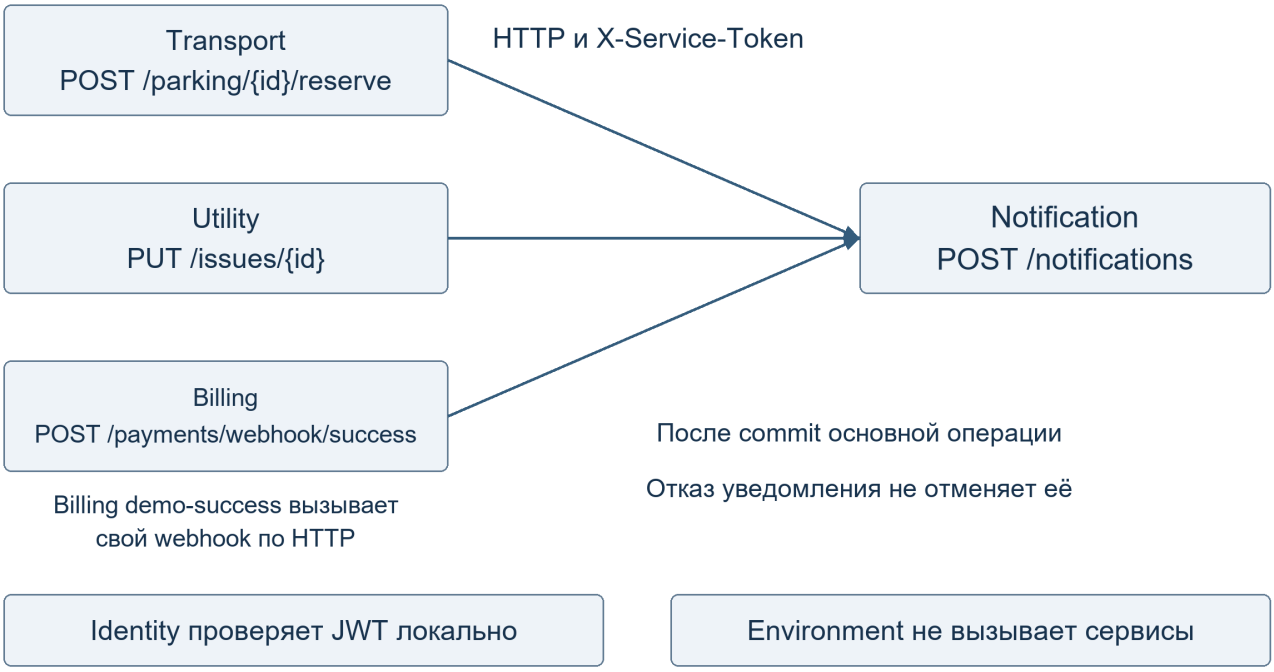
Цель и полученный результат

Мы объединили Utility и Notification в одно приложение с одной SQLite-базой, сохранили внешние REST API и проверили работу всего исходного города с заменой этой пары. В выбранном сценарии сетевой запрос Utility → Notification заменён синхронным вызовом публичного фасада. Исходная версия сохранена в том же репозитории и запускается независимо.

В наших трёх прогонах средняя задержка выбранной пары составила 411.29 мс у микросервисов и 533.84 мс у монолита. Полезная работа подтверждена для каждого запроса. Эти результаты относятся к конкретным реализациям и условиям ниже; они не являются универсальной оценкой двух архитектур. Монолит потребовал меньше памяти, но в этой серии отвечал медленнее.

Исходный проект и карта взаимодействий

Источник — SmartCity на зафиксированном commit. В `smart-city/` сохранены все 132 отслеживаемых файла приложения. Бизнес-код шести сервисов и React frontend совпадает с источником с учётом LF/CRLF; преднамеренные отличия конфигурации описаны в `docs/SMARTCITY_ORIGIN.md`.



Фактическая карта исходящих серверных вызовов

Источник и входной endpoint	Получатель	Поведение при отказе Notification
Transport, POST /parking/{id}/reserve	POST /notifications после новой брони	Бронь и списание места сохраняются, клиент получает 201, ошибка логируется
Utility, PUT /issues/{id}	POST /notifications при смене статуса	Новый статус сохраняется, клиент получает 200, ошибка логируется
Billing, POST /payments/webhook/success	POST /notifications после первого подтверждения	Платёж, счёт и событие сохраняются, клиент получает 200, ошибка логируется

Все три вызова используют `X-Service-Token`, HTTPX и тайм-аут 3 секунды. Billing также вызывает собственный webhook из `POST /payments/{id}/demo-success`. Это внутрисервисный HTTP-вызов; при его ошибке клиент получает 502, а сохранённое состояние следует проверить безопасным повтором. Identity, Environment и Notification исходящих серверных вызовов не выполняют. JWT проверяется локально, без HTTP-запроса к Identity. Браузер обращается к API отдельно; такие обращения не считаются межсервисными вызовами.

Выбранная пара и сценарий

Utility и Notification выбраны потому, что взаимодействие уже реализовано в исходном городе и имеет проверяемый побочный эффект. Смена статуса заявки не требует платёжного провайдера или расходования парковочных мест, поэтому подходит для повторяемого сравнения.

1. Пользователь с ролью USER создаёт новую заявку через `POST /issues`; в БД сохраняется `NEW`.

2. OPERATOR или ADMIN отправляет `PUT /issues/{id}` с `{"status":"IN_PROGRESS"}`.

3. Utility выполняет условный UPDATE только при отличающемся статусе и фиксирует заявку.

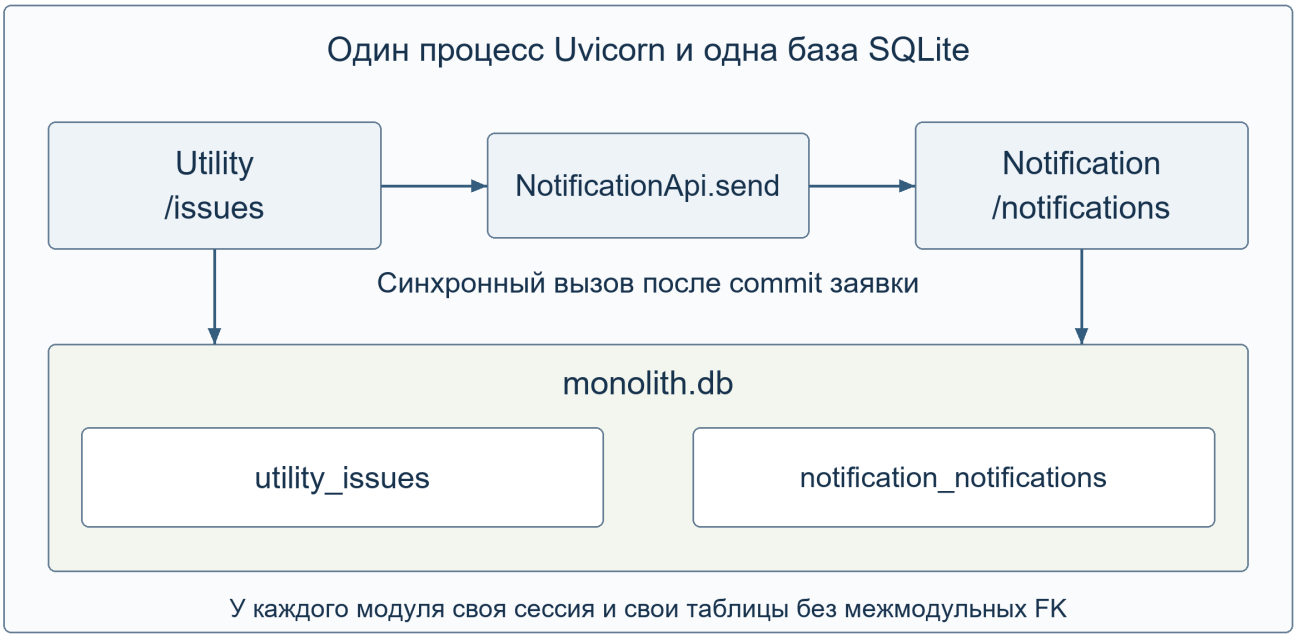
4. В micro Utility вызывает HTTP `POST /notifications`. В mono он вызывает `NotificationApi.send` напрямую.

5. Notification в собственной сессии фиксирует `PENDING`, имитирует отправку и отдельно фиксирует `SENT` с UTC `sent_at`.

6. Клиент получает прежний JSON заявки с кодом 200. При сбое отправки заявка остаётся сохранённой, а уведомление — `PENDING`.

Выбран вариант A, прямой синхронный вызов. Он сохраняет порядок исходных действий и позволяет сравнить одинаковую полезную работу. Внутренние события для этой однонаправленной связи добавили бы обработчик и правила доставки, которые не нужны для поставленного эксперимента.

Модули и единая база данных



Модули монолита и их данные

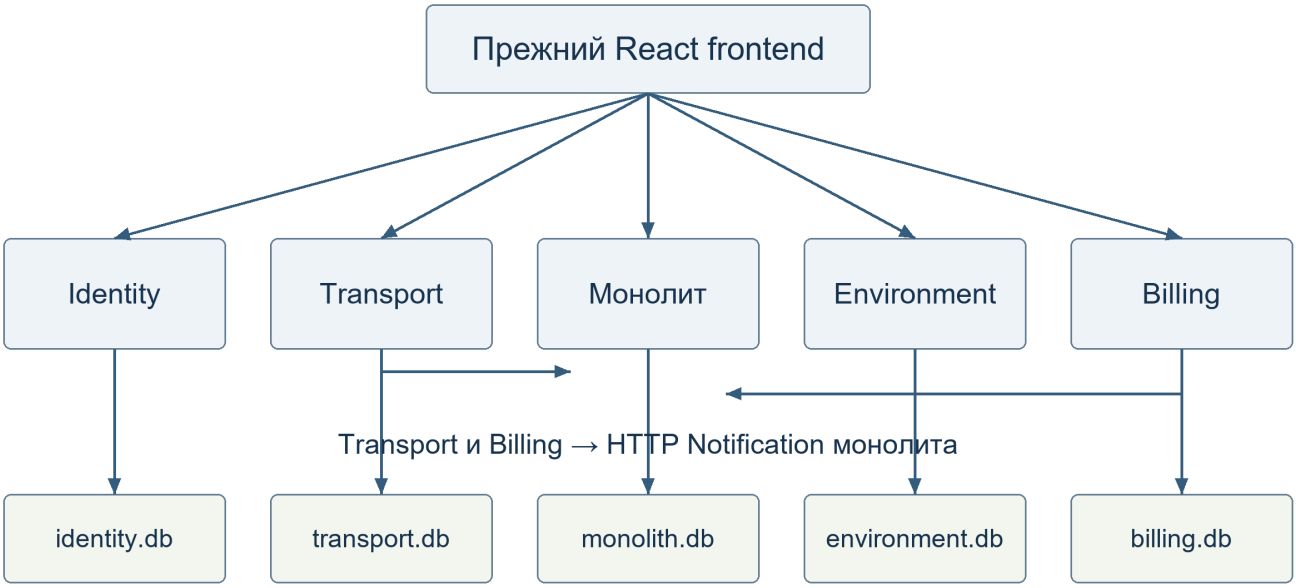
`app/main.py` собирает два APIRouter без новых префиксов. Общие engine, Base и SessionLocal находятся в `app/database.py`; `get_notification_api()` предоставляет настроенный фасад. Utility импортирует Notification только через `app.modules.notification.api`. Фасад возвращает публичный Pydantic DTO, валидирует UUID, channel и ограничения строк, сам открывает и закрывает сессию. Обратной зависимости, межмодульных JOIN и обращения к чужим ORM-моделям нет.

Сохранён исходный стек: Python 3.12, FastAPI 0.115.12, Uvicorn 0.34.2, синхронный SQLAlchemy 2.0.40, PyJWT 2.10.1 и SQLite через встроенный драйвер. В micro два файла SQLite и два процесса приложения; в mono один файл `monolith.db`, таблицы `utility_issues` и `notification_notifications`, один процесс Uvicorn с одним worker. Остальные сервисы имеют собственные БД.

В задании приведены схемы PostgreSQL и префиксы MySQL. Мы сохранили исходную SQLite и применили префиксы таблиц как аналогичное разделение модульных данных. Это явно выбранная трактовка раздела о единой БД, а не использование PostgreSQL-схем. WAL и отдельный сервер СУБД не добавлены. Скрипт `python -m app.init_db` создаёт структуру и демонстрационные записи; повторный запуск сохраняет существующие данные.

Внешний ключ между модулями не создан. Он мог бы проверять существование связанной записи на уровне БД, но связывал бы миграции и жизненные циклы модулей и мешал их последующему выделению. В текущей модели отдельного `issue_id` у уведомления нет: идентификатор заявки включён в текст. Поэтому нельзя утверждать, что БД проверяет связь уведомления с заявкой. За корректность передаваемых данных отвечает публичный интерфейс.

Совместимость полного города



Полный город 7 → 6 контейнеров с frontend Измеряемая пара 2 → 1

Целевой SmartCity с четырьмя прежними сервисами

Сохранены `POST/GET /issues`, `PUT /issues/{id}`, роли и JWT, коды 201/200/409 при создании и повторе заявки, формат ошибок и UTC-даты с `Z`. Уникальность `(user_id, request_key)` защищает от гонки POST; условный UPDATE не допускает второго уведомления при двух одинаковых сменах статуса. `POST /notifications` сохранён, поскольку его по HTTP используют Transport и Billing. Он защищён служебным токеном и при прямом сбое возвращает 500.

Внешние порты Utility и Notification по умолчанию 8003 и 8006 отображаются на один порт 8000 монолита. Frontend сохранён, адреса API задаются в Compose. Полный исходный город содержит 7 контейнеров с frontend, целевой — 6. Это состав конфигурации, а не измерения памяти всей городской системы.

Прошли 67 модульных, инфраструктурных и сквозных тестов, включая отказ, идемпотентность и конкурентные запросы; отдельно прошли 5 статических проверок Compose. В обоих полных стендах через HTTP проверены вход USER/OPERATOR/ADMIN, транспорт, парковки, датчики, счета, создание и обновление заявки, новая бронь, демо-оплата и соответствующие SENT-уведомления. Проверены CORS, оба адреса монолита, защита Notification и отдача frontend с обработкой JSX Vite. Это автоматические API-проверки; порядок интерактивного показа приведён ниже.

Условия и методика измерений

Измерена только подсистема Utility + Notification. Полные города остановлены; micro и mono запускались последовательно на одном компьютере. Окружение: Windows с WSL2, Ubuntu 20.04.6 LTS (containerized), Docker 26.1.3, CPU AMD Ryzen 7 5800X 8-Core Processor, 16 логических CPU, доступная Docker память 15.54 ГиБ. Файлы обеих версий размещены на одном NVMe SK hynix BC501, NTFS через одинаковые bind-backed named volumes WSL. Это отдельные новые тестовые данные. Во всех трёх API-контейнерах Python 3.12.15, SQLite 3.46.1; общие пакеты совпадают по версиям. `journal_mode=delete`, `busy_timeout=30000`, `foreign_keys=1`, `synchronous=2`. Точные версии, параметры и команды процессов сохранены в JSON.

Первоначальный прогон на хранилище WSL/HDD был отклонён: все 200 заявок изменились, но только 148 получили SENT, а 52 HTTP-вызова Notification завершились ReadTimeout. Диагностика обнаружила commit порядка 140–150 мс; на NVMe через тот же Docker путь — около 16 мс. Для принятой серии обе версии перенесены на одинаковый NVMe без изменения тайм-аута HTTP 3 секунды, режима журнала или synchronous. Невалидный прогон не включён в средние; очищенное описание диагностики сохранено в `results/storage-check.json`.

Код приложения и инструмент сравнения зафиксированы на commit `9e940616bff9e7f415e650fd95d9d6bad5c6e4ae`. Из-за отсутствия прямого сетевого доступа WSL образы были собраны офлайн из копий того же кода, официальных базовых образов и пакетов с общими constraints. Обоим стендам предоставлен один и тот же набор пакетов; настройки БД и логирования не менялись. Локальные кэши, образы и зависимости не входят в репозиторий.

Для каждой версии выполнено 3 прогона. Перед каждым через API созданы 200+20 новых заявок: 20 для прогрева и 200 для измерений. Каждый измеряемый PUT однократно меняет отдельную заявку `NEW → IN_PROGRESS`, concurrency=10. Подготовка, JWT, прогрев и чтение БД исключены из времени. Использован один инструмент `benchmarks/run.py`, HTTPX с одинаковыми keep-alive и тайм-аутами.

Средняя задержка — среднее индивидуальных длительностей запросов без времени ожидания в очереди клиента. RPS = 200 / длительность всего пакета. После каждого прогона read-only проверка подтвердила 200 изменённых заявок и 200 новых SENT с точным текстом, автором и получателем; прогрев исключён. Ошибочных ответов и неподтверждённых уведомлений в принятых прогонах нет.

Во время нагрузки сборщик вызывал `docker stats --no-stream` для обоих micro-контейнеров или единственного mono-контейнера. В среднее включены только снимки, полностью полученные внутри измеряемого пакета, минимум два на прогон. Память суммируется по приложениям и усредняется арифметически; 1 МБ = 10^6 байт, 1 МиБ = 2^20 байт. Docker показывает рабочий набор памяти, а не полную память Windows/WSL. SQLite встроена в процессы; отдельного контейнера БД нет. Исходные строки и реальные интервалы снимков сохранены. Один и тот же генератор нагрузки запускался отдельным служебным контейнером; его память и контейнер не включены в показатели приложений ни одной версии.

Результаты сравнения

Показатель выбранной пары	Микросервисы	Модульный монолит
Среднее время ответа, мс	411.29	533.84
Запросов в секунду	23.98	18.62
Средняя память приложений, МБ	182.53	62.77
Контейнеров приложений	2	1
Команд запуска готового стенда	1	1

Версия и прогон	Среднее время, мс	Запросов/с	Память, МБ	Снимков под нагрузкой
Микросервисы 1	484.88	20.07	175.69	3
Микросервисы 2	387.84	25.04	184.17	2
Микросервисы 3	361.13	26.81	187.75	2
Микросервисы среднее	**411.29**	**23.98**	**182.53**	-
Монолит 1	470.94	20.76	61.92	3
Монолит 2	480.51	20.19	62.98	3
Монолит 3	650.07	14.91	63.42	5
Монолит среднее	**533.84**	**18.62**	**62.77**	-

Средние итоговой таблицы рассчитаны по трём значениям каждого показателя. Команд запуска готового стенда по одной: `docker compose up -d --wait` с нужным Compose-файлом. Подготовка зависимостей и проверочные команды в этот показатель не включены у обеих версий.

В этом эксперименте отношение задержки micro/mono равно 0.77, а память монолита относительно суммы micro изменилась на -65.61%. Средняя задержка монолита выросла на 29.80%, а RPS изменился на -22.34%. Поэтому результаты не подтверждают ускорение монолита. Разницу нельзя полностью приписать устранению HTTP: одновременно меняются число процессов, файлов SQLite и конкуренция за одного писателя. Три прогона на одной машине показывают наблюдаемую тенденцию, но не заменяют проверку других нагрузок и статистическую оценку на большем числе запусков.

Эксперимент с отказом

В обеих версиях `NOTIFICATION_FAIL_SEND=true` включает RuntimeError после commit PENDING и до присвоения SENT. Использованы новые заявки, а не повтор прежнего статуса. Ошибка обработки подтверждена HTTP и чтением сохранённых данных из отдельного соединения.

Наблюдение	Микросервисы	Монолит
Прямой POST /notifications с корректным токеном	500	500
PUT заявки при отказе Notification	200, IN_PROGRESS	200, IN_PROGRESS
Сохранённое уведомление	PENDING, sent_at=NULL	PENDING, sent_at=NULL
Тот же статус после выключения отказа	Нового уведомления нет	Нового уведомления нет
Другая новая заявка после выключения отказа	SENT, sent_at заполнен	SENT, sent_at заполнен
Прежнее PENDING после восстановления	Осталось PENDING	Осталось PENDING

Best-effort уже присутствовал в исходных микросервисах. Монолит сохраняет его: Utility фиксирует заявку до обращения к фасаду и перехватывает исключения только вокруг отправки; ошибки самой операции с заявкой выходят наружу. Отказ уведомления не отменяет основную операцию, но доставка не гарантируется. Для гарантированной доставки потребовались бы durable outbox, повторные попытки и защита от дублирования. В этой работе они не реализованы.

Выводы

Для выбранного сценария результаты подтверждают уменьшение числа развёртываемых приложений и суммарной памяти после объединения пары. При этом монолит оказался медленнее по средней задержке и RPS. Возможное объяснение — конкуренция трёх фиксаций за одного SQLite-писателя вместо двух отдельных файлов; эксперимент не изолирует этот фактор от остальных. Переносить эти числа на весь SmartCity нельзя. Прямой вызов упрощает локальную диагностику и убирает сетевую границу между двумя модулями.

Микросервисы сохраняют независимое развёртывание, отдельные файлы БД и возможность масштабировать Notification отдельно. За это приходится платить дополнительным процессом, HTTP-взаимодействием и обслуживанием нескольких компонентов. У монолита общее развёртывание и один SQLite-писатель; сбой всего процесса затрагивает оба модуля, а границы необходимо поддерживать кодом и проверкой зависимостей.

Для учебного города с одной командой и умеренной нагрузкой мы выбрали бы реализованное объединение Utility + Notification, сохраняя остальные сервисы как отдельные компоненты: экономия памяти здесь важнее максимального RPS рассмотренного сценария. При строгом требовании к задержке эти измерения дают основание оставить пару отдельной и дополнительно изучить узкое место. Такой гибрид подходит, когда части системы имеют разные требования к нагрузке и частоте выпуска. При необходимости независимого масштабирования или работы нескольких команд Notification можно снова выделить через его публичный интерфейс. Неожиданным оказалось, что устранение HTTP не обеспечило ускорения, а устойчивость заявки к отказу уведомления уже обеспечивал исходный код: смена архитектуры сама по себе не создаёт гарантированной доставки и не исправляет PENDING.

Воспроизведение и показ

Команды выполняются из `smart-city-monolith/`, если не указано иное. Тесты: `python -m pytest -q`; проверки Compose из корня: `python -m unittest discover -s tests -v`. Создание структуры: `python -m app.init_db` на отдельной БД приложения.

1. Из корня запустить целевой город: `docker compose up -d --build --wait`. Если порты заняты, задать свободный диапазон через переменные из `.env.example`, не останавливая чужой стек.

2. Открыть прежний frontend, войти демонстрационным пользователем, создать заявку; оператором изменить её статус. Показать прежний JSON и новые записи в двух таблицах монолита.

3. Показать остальные страницы города, новую бронь и демо-оплату, затем соответствующие уведомления от Transport/Billing.

4. На изолированном стенде включить отказ, изменить статус новой заявки и показать 200, сохранённую заявку и PENDING/NULL. Выключить отказ и использовать другую новую заявку для SENT.

5. Открыть таблицу измерений и JSON с исходными длительностями, снимками памяти и проверками побочного эффекта.

Команды изолированных стендов и отказа приведены в `benchmarks/ORIGIN.md`. Инструменты: `python benchmarks/run.py --help` и `python benchmarks/failure.py --help`. Доказательства — `smart-city-monolith/benchmarks/results/`: `micro.json`, `mono.json`, `micro-failure.json`, `mono-failure.json`, `hybrid-smoke.json`, `original-smoke.json`, `summary.csv`. Основной редактируемый отчёт — этот Markdown; PDF является экспортом того же содержания.